Universidad de San Carlos de Guatemala
Escuela de Ingeniería en Ciencias y Sistemas, Facultad de Ingeniería
Introducción a la programación y computación 2, 2do. Semestre 2026

Proyecto IPC2: API de Autorización SAT

202209480 – Ronald Daniel Osorio Chávez

Resumen

El presente artículo detalla el diseño y desarrollo de una Interfaz de Programación de Aplicaciones (API) orientada a la autorización de Documentos Tributarios Electrónicos (DTE) para la Superintendencia de Administración Tributaria (SAT). En el contexto actual, la validación estricta y rápida de datos tributarios masivos representa un desafío crítico para la computación algorítmica y los tiempos de respuesta de los servidores web.

Para resolverlo, el proyecto implementa un backend utilizando el framework .NET Core y lenguaje C#, aplicando el paradigma de Programación Orientada a Objetos. Se integran estructuras de memoria eficientes basadas en diccionarios Hash para garantizar búsquedas en tiempo $O(1)$, así como algoritmos de validación matemática (Módulo 11). Finalmente, la arquitectura se divide en un esquema cliente-servidor interactuando mediante payloads en formato JSON y peticiones HTTP.

Palabras clave

Palabras clave: API REST, C#, .NET Core, Razor Pages, Módulo 11, JSON, HTTP.

Abstract

This article details the design and development of an Application Programming Interface (API) focused on the authorization of Electronic Tax Documents (DTE) for the Superintendency of Tax Administration (SAT). In the current context, strict and fast validation of massive tax data represents a critical challenge for algorithmic computing and web server response times.

To solve this, the project implements a backend using the .NET Core framework and C# language, applying the Object-Oriented Programming paradigm. Efficient memory structures based on Hash dictionaries are integrated to ensure search times in $O(1)$, as well as mathematical validation algorithms (Modulo 11). Finally, the architecture is divided into a client-server scheme interacting through JSON payloads and HTTP requests.

Keywords

Keywords: REST API, C#, .NET Core, Razor Pages, Modulo 11, JSON, HTTP.

Introducción

La administración de transacciones tributarias masivas exige sistemas capaces de procesar, validar e interpretar datos complejos en tiempo real. La eficiencia de estas interfaces de servicio (API) depende directamente de cómo se modela y gestiona la memoria subyacente. El propósito de este ensayo

es analizar la arquitectura del sistema de autorización de facturas desarrollado, evidenciando cómo la separación de responsabilidades entre cliente y servidor optimiza el flujo de trabajo.

A través de este documento, se responderá cómo la implementación de Diccionarios y Conjuntos Hash en memoria supera exitosamente las validaciones algorítmicas requeridas por el Registro Tributario Unificado (RTU). Los aportes técnicos y arquitectónicos presentados sirven como base para comprender la superioridad de las estructuras dinámicas $O(1)$ frente a las búsquedas lineales secuenciales.

Desarrollo del tema

a. **Arquitectura Cliente-Servidor:** El sistema fue desplegado utilizando un esquema distribuido. El cliente web (Frontend), desarrollado bajo ASP.NET Razor Pages, actúa como un consumidor que envía archivos JSON mediante el protocolo HTTP. Por su parte, la API REST (Backend) concentra la lógica de negocio, recibiendo cargas de trabajo, serializando los objetos mediante `System.Text.Json` y respondiendo con códigos de estado estandarizados.

b. **Gestión de Memoria y Complejidad O(1):** Para garantizar tiempos de respuesta casi instantáneos incluso ante altos volúmenes de validaciones simultáneas, el Registro Tributario Unificado (RTU) se modeló en memoria utilizando la colección genérica `Dictionary<string, string>`. Esto permite acceder, actualizar e insertar contribuyentes sin requerir recorridos iterativos. De igual forma, se utilizó un `HashSet` para almacenar las referencias únicas de las facturas aprobadas, garantizando una prevención de duplicados con complejidad temporal constante.

c. **Motor de Validación Módulo 11:** El núcleo del sistema radica en la validación estricta de las reglas del negocio, específicamente para los Números de Identificación Tributaria (NIT). El algoritmo iterativo implementado analiza la cadena en orden inverso, ponderando posicionalmente cada dígito para extraer el residuo módulo 11 exacto, descartando instantáneamente transacciones de contribuyentes inválidos antes de impactar la base de datos temporal.

d. **Generación Dinámica de Correlativos:** La emisión de un código único de 16 posiciones (`yyyymmdd########`) requirió un módulo especializado que consolida las aprobaciones diarias. Utilizando Expresiones Regulares (Regex) para depurar fechas de entrada, el sistema administra contadores diarios independientes, rellenando con ceros a la izquierda para cumplir estrictamente con el formato estandarizado requerido por la institución.

Conclusiones

1. La separación de la interfaz gráfica y el motor lógico en dos servicios independientes (Frontend y API Backend) permite modularizar el proyecto, garantizando que futuras expansiones del servicio de autorización de facturas no comprometan el rendimiento de la aplicación web de cara al usuario final.

2. La implementación de colecciones genéricas avanzadas como los `Dictionary` y `HashSet` resultan indispensables para el desarrollo de APIs tributarias, ya que superan ampliamente las limitaciones de velocidad inherentes a las listas simples, asegurando el procesamiento de miles de JSONs en fracciones de segundo.

Referencias bibliográficas

I. Microsoft Corporation. (2026). *C# Documentation - ASP.NET Core Web API y Procesamiento JSON*. Microsoft Learn.

II. Superintendencia de Administración Tributaria. (2026). *Definición y Algoritmo de validación de NIT (Módulo 11) para DTE*.

III. Freeman, A. (2022). *Pro ASP.NET Core 6: Develop Cloud-Ready Web Applications Using MVC, Blazor, and Razor Pages*. Apress.

Diagrama de Despliegue

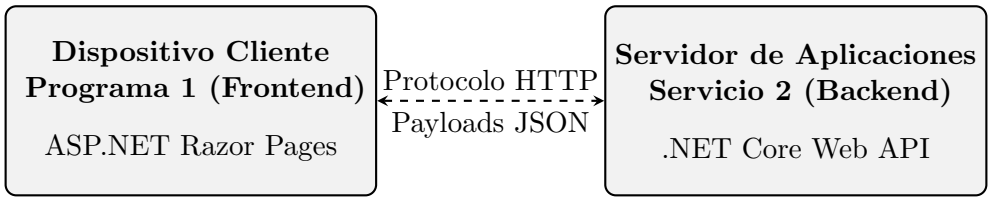


Figura 1: Diagrama de despliegue mostrando la arquitectura de comunicación entre Frontend y Backend.

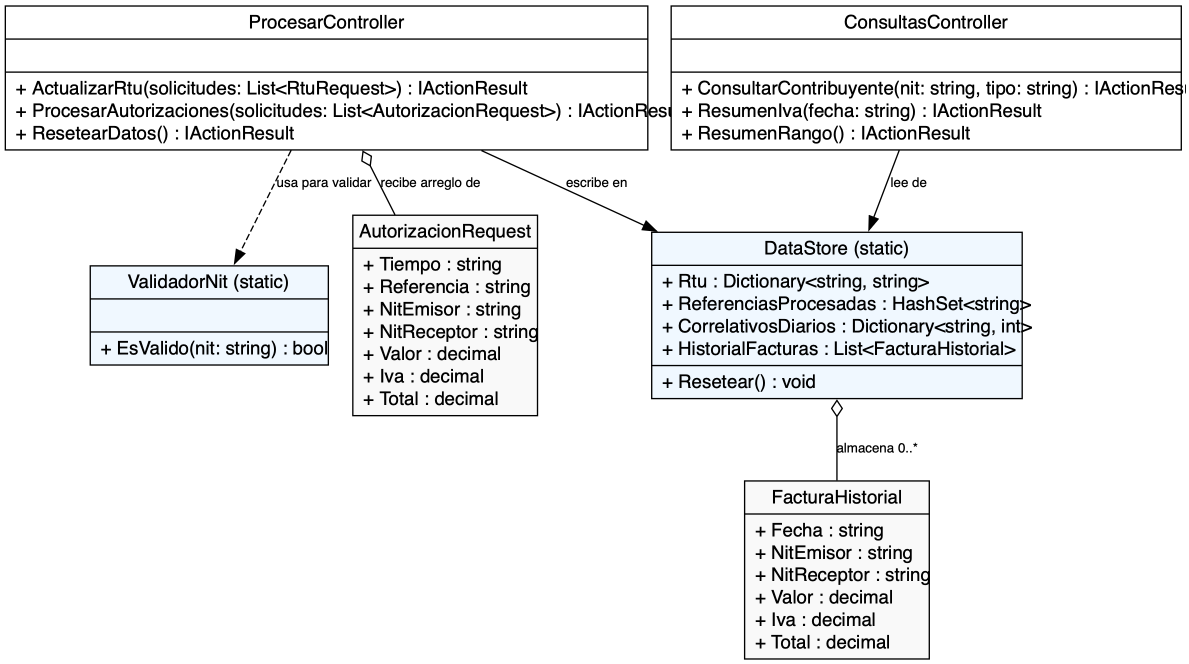


Figura 2: Diagrama de clases representando la estructura de datos y relaciones entre las entidades del sistema de autorización de facturas.